

Credit Card Approval Prediction

AI-Powered Credit Risk Assessment & Approval Prediction System

Team ID: CCAP-01 | Date: 02 July 2026

Project Description:

Credit Approval AI is an end-to-end machine learning web application that predicts whether a credit card application should be Approved or Rejected, based on applicant demographics, income, employment, housing, and credit history signals. The platform takes an applicant record — either through a web form for a single applicant or a CSV upload for batch scoring — and returns an approval decision along with an approval probability, a confidence score, a risk meter (Low / Moderate / High), and a short list of decision reasons.

The system is built around a reusable data pipeline that merges application and credit history records, cleans missing values and outliers, encodes categorical fields, engineers features such as Age Group, Income Category, Employment Experience, Family Income Ratio, and Financial Stability Score, and then trains and compares multiple classification algorithms to select the best-performing model. The trained pipeline is served through a Flask web application with user authentication (including Google Sign-In), a prediction dashboard, an admin analytics panel, and PDF/Excel export of results.

Scenarios

Scenario 1: A bank employee enters a single applicant's details — age, income, occupation, education, housing type, and family size — into the Prediction form. The system instantly returns an Approved or Rejected decision with an approval probability, a risk category, and the top reasons behind the decision, which can be exported as a PDF or Excel report.

Scenario 2: A credit operations team needs to screen a batch of new applicants at once. They upload a CSV file containing the required applicant columns, and the app validates the columns, runs every row through the trained model, and returns a downloadable CSV of predictions with approval status and risk level for each applicant.

Scenario 3: An administrator logs into the Admin dashboard to review recent decisions. The Home page shows saved prediction counts, approved-vs-rejected totals, and a risk-level breakdown as interactive charts, letting the team monitor approval trends and delete or clear historical predictions as needed.

Technical Architecture:

Credit Approval AI follows a modular three-tier architecture. The frontend is a set of Flask-rendered HTML templates (home, prediction, admin, about, model, features, login, signup) that let users submit applicant data and view results. The Flask backend (app.py) handles routing, session-based authentication, form validation, SQLite-backed prediction history, and file handling for CSV batch uploads and PDF/Excel exports. It delegates all machine-learning logic to a dedicated prediction module (predict.py) built on a preprocessing pipeline (preprocessing.py) that mirrors the feature engineering used during training.

Layer	Component	Responsibility
Frontend	Jinja2 templates + CSS/JS	Home, Prediction, Admin, About, Model, Features, Login, Signup pages
Backend	Flask (app.py)	Routing, auth (local + Google OAuth), validation, history, PDF/Excel export
ML Layer	predict.py + preprocessing.py	Feature engineering, model loading, single & batch prediction
Storage	SQLite (prediction_history.db, users)	Prediction history, user accounts, admin analytics

Pre-requisites:

- Flask Framework Knowledge — routing, sessions, templates, request handling
- Python & Data Science Libraries — pandas, numpy, scikit-learn, xgboost, joblib
- HTML, CSS, and JavaScript Skills for building the Flask-rendered UI
- SQLite basics for the prediction history and user accounts tables
- Google OAuth 2.0 basics for the "Continue with Google" login option
- reportlab and openpyxl for PDF and Excel report generation
- Version Control with Git, and gunicorn for production-style deployment

Project Workflow:

Milestone 1: Model Selection and Architecture

- Activity 1.1: Collect and merge application_record and credit_record datasets by applicant ID.
- Activity 1.2: Research and compare candidate classification models for approval prediction.
- Activity 1.3: Define the application architecture — Flask backend, preprocessing pipeline, model layer.
- Activity 1.4: Set up the development environment and install dependencies from requirements.txt.

Milestone 2: Core Functionalities Development

- Activity 2.1: Build the data-cleaning and feature-engineering pipeline (missing values, outliers, encoding, engineered features).
- Activity 2.2: Implement single-applicant prediction, batch CSV prediction, and result export in the Flask backend.

Milestone 3: App.py Development

- Activity 3.1: Write the main Flask application logic — routes, authentication, validation, and history management.

Milestone 4: Frontend Development

- Activity 4.1: Design the multi-page UI — Home, Prediction, Features, About, Model, Admin, Login, Signup.
- Activity 4.2: Build dynamic dashboards for prediction history, approval/rejection charts, and risk levels.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment and verify all routes end-to-end.
- Activity 5.2: Package the app for production-style deployment using gunicorn.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

This milestone focuses on preparing the raw credit data and choosing an architecture and model family capable of predicting approval outcomes reliably from demographic and credit-history signals.

Activity 1.1: Data Collection

The project uses two source files, `application_record.csv` and `credit_record.csv`. Both are merged by the applicant ID, and historical repayment statuses in `credit_record.csv` are converted into a binary target column that marks each applicant as a good or bad credit risk, which the model is trained to predict as Approved or Rejected.

Activity 1.2: Research and Select the Modelling Approach

- Understand the Requirements: review the target column definition, class balance, and the demographic/financial fields available for prediction.
- Compare Candidate Algorithms: Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, KNN, Naive Bayes, and Gradient Boosting, with XGBoost evaluated as an optional stronger learner (all listed in `requirements.txt` via `scikit-learn` and `xgboost`).
- Evaluate Performance: compare models on accuracy, precision/recall, and ROC-AUC to balance catching risky applicants against not over-rejecting good ones.
- Select and Persist the Best Pipeline: the best-performing model and its metadata are saved (loaded later in `app.py` from `models/model_metadata.pkl`) alongside a `model_comparison.csv` report used to populate the Model page.

Activity 1.3: Define the Application Architecture

- Frontend: Jinja2 templates rendered by Flask for Home, Prediction, Admin, About, Model, Features, Login, and Signup.
- Backend: `app.py` handles all HTTP routes, session-based login, Google OAuth, form/CSV validation, and history persistence.
- ML Integration: `predict.py` exposes a `CreditApprovalPredictor` class used by `app.py` for both single-record and dataframe (batch) predictions, backed by the shared preprocessing pipeline in `preprocessing.py`.

Activity 1.4: Set Up the Development Environment

Install Python 3.11+, create a virtual environment, and install the project dependencies:

```
python -m venv .venv
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

requirements.txt (actual project dependencies):

```
Flask==3.0.3           python-dotenv==1.0.1
pandas==2.2.3          numpy==2.0.2
scikit-learn==1.5.2    joblib==1.4.2
xgboost==2.1.3         shap==0.46.0
matplotlib==3.9.4     seaborn==0.13.2
plotly==5.24.1         reportlab==4.2.5
openpyxl==3.1.5        gunicorn==23.0.0
pytest==8.3.4
```

Milestone 2: Core Functionalities Development

Activity 2.1: Feature Engineering & Preprocessing

The preprocessing pipeline (`preprocessing.py`) transforms raw applicant fields into model-ready features, several of which are surfaced directly on the Features page of the site:

- **Age Group:** groups customers into young, early career, mid career, senior, and retired ranges.
- **Income Category:** transforms annual income into low, moderate, high, and very high bands.
- **Employment Experience:** converts employment days into practical years employed.
- **Family Income Ratio:** measures income support available per family member.
- **Children Category:** summarizes dependent count into no children, small family, or large family.
- **Working Status:** separates employed customers from pensioner or non-working profiles.
- **Housing Ownership:** captures whether the housing situation looks stable.
- **Financial Stability Score:** combines ownership, employment, and income strength into a score.
- **Risk Category:** converts the stability score into low, medium, or high risk.

Activity 2.2: Prediction & Export Functionalities

- **Single Prediction:** `validate_form()` checks required applicant fields (name, phone, gender, age, income, family size, education, occupation, housing type, ownership flags, years employed, children, income type, marital status) before the record reaches the model.
- **Batch Prediction:** the `/batch` route accepts a CSV, checks it against `BATCH_REQUIRED_COLUMNS` (ID, CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, CNT_CHILDREN, AMT_INCOME_TOTAL, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE, DAYS_BIRTH, DAYS_EMPLOYED, OCCUPATION_TYPE, CNT_FAM_MEMBERS), and returns a downloadable predictions CSV.
- **Decision Reasons:** `prediction_reasons()` explains each decision in plain language — e.g. income level, employment stability, ownership signals, dependants, and overall probability — capped to the five most relevant reasons.
- **History & Analytics:** every prediction is saved to a SQLite `prediction_history` table; `history_stats()` aggregates totals, approved/rejected counts, and risk-level counts for the Home page charts and Admin dashboard.
- **Report Export:** `/download-pdf` (via `reportlab`) and `/download-excel` (via `openpyxl`) generate a one-page PDF or an Excel workbook summarizing a prediction result.

Milestone 3: App.py Development

Milestone 3 covers the actual Flask application logic in app.py — the routes, authentication, and prediction handling that power every page of the site.

Core Routes

Home route — renders the dashboard with live stats and recent history:

```
@app.route("/")
def home():
    predictor = get_predictor()
    predictor_ready = predictor is not None
    stats = history_stats()
    return render_template(
        "index.html", predictor_ready=predictor_ready,
        history=recent_history(), stats=stats,
    )
```

Prediction route — validates the form, runs the model, and saves the result:

```
@app.route("/predict", methods=["GET", "POST"])
@login_required
def prediction():
    if request.method == "GET":
        return render_template("prediction.html", result=None, form_data={})
    errors = validate_form(request.form)
    if errors:
        for error in errors: flash(error, "danger")
        return render_template("prediction.html", result=None, form_data=request.form)
    predictor = get_predictor()
    result = predictor.predict_form(request.form)
    result.reasons = prediction_reasons(request.form, result)
    save_history(request.form, result)
    return render_template("prediction.html", result=result, form_data=request.form)
```

Batch prediction route — validates CSV columns and scores every row:

```
@app.route("/batch", methods=["POST"])
@login_required
def batch_prediction():
    predictor = get_predictor()
    file = request.files.get("csv_file")
    raw = pd.read_csv(upload_path)
    missing_columns = missing_batch_columns(raw)
    if missing_columns:
        flash("Your CSV is missing required columns: " + ", ".join(missing_columns), "danger")
        return redirect(url_for('prediction'))
    predictions = predictor.predict_dataframe(raw)
    predictions.to_csv(output_path, index=False)
    return render_template("batch_results.html", rows=predictions.head(50).to_dict("records"),
        columns=predictions.columns.tolist(), total=len(predictions))
```

Authentication Routes

Local login/signup with SQLite-backed users, plus Google OAuth as an alternate sign-in path:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    user = find_user(username)
    if (user and check_password_hash(user["password_hash"], password)) or (
        username == expected_username and password == expected_password
    ):
        session["logged_in"] = True
        return login_redirect_target()

@app.route("/login/google")
def google_login():
    # Builds the Google OAuth authorization URL with a CSRF state token
    return redirect(f"{GOOGLE_AUTH_URL}?{urllib.parse.urlencode(params)}")
```

Full Route Map

- / — Home dashboard with live stats and recent predictions
- /login, /signup, /login/google, /login/google/callback, /logout — authentication
- /predict — GET renders the form, POST validates & returns a prediction
- /batch, /batch/download/<filename>, /sample-batch-csv — CSV batch scoring
- /download-pdf, /download-excel — export a single prediction as a report
- /about, /features, /model, /contact — informational pages
- /admin — analytics dashboard (login required)
- /history/delete/<id>, /history/clear — manage saved prediction history

Milestone 4: Frontend Development

Milestone 4 focuses on the multi-page Flask front end. Each page is a Jinja2 template sharing a common navigation bar (Home, Prediction, Admin, About, Model, Features, Contact) and a Dark Mode toggle, with authenticated users seeing Logout instead of Login/Sign Up.

Activity 4.1: Page Layout & Components

- Home Page: hero section, live "Application Desk" overview (saved predictions, approved/rejected counts, batch-ready status), a 5-step workflow (Collect, Prepare, Train, Predict, Report), and recent prediction history with Approved/Rejected badges and risk labels.
- Prediction Page: an Applicant Details form (name, gender, age, income, family members, children, years employed, education, occupation, housing type, income type) alongside a result-preview panel and a CSV batch-upload option.
- About Page: Problem Statement, Objective, Dataset, and Algorithms cards describing the ML approach in plain language.
- Features Page: nine cards documenting each engineered feature used by the model.
- Login / Signup Pages: local email-or-phone login with a password, a "Continue with Google" OAuth option, and a signup form for creating a local account.

Activity 4.2: Dashboards & Visual Feedback

- Recent Prediction History table with Time, Name, Income, Education, Prediction, Probability, Risk, and a Delete action per row.
- Approved vs Rejected donut chart and a Risk Levels bar chart (High / Low / Moderate Risk) rendered from `history_stats()`.
- Clear History and New Prediction actions for managing the dashboard.

Milestone 5: Deployment

Activity 5.1: Local Deployment

With the virtual environment active and dependencies installed, configure environment variables in a .env file (SECRET_KEY, FLASK_ENV, and optionally GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REDIRECT_URI for Google Sign-In), then run the app locally:

```
python app.py
* Serving Flask app 'app'
* Running on http://127.0.0.1:5000
```

Test every route end to end — signup and login (including Google OAuth, if configured), single prediction, CSV batch prediction, PDF/Excel export, admin dashboard, and history delete/clear.

Activity 5.2: Production-Style Deployment

For a production-style run, the app is served with gunicorn instead of the Flask development server, and debug mode is left off so errors are handled by the custom 404/500 templates:

```
gunicorn app:app --bind 0.0.0.0:8000 --workers 3
```

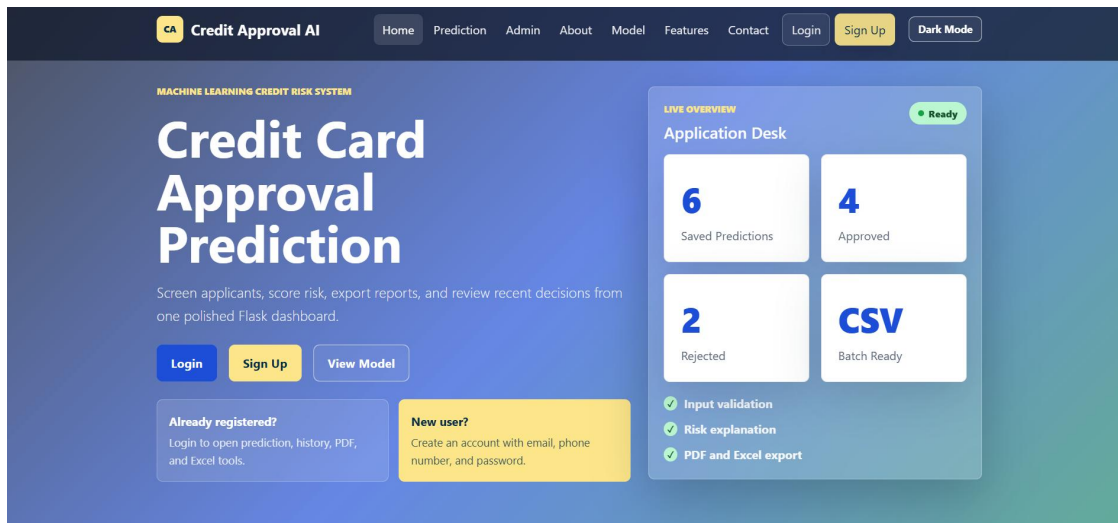
Important Notes:

- SECRET_KEY must be set to a long random value in .env before any real deployment.
- GOOGLE_REDIRECT_URI must exactly match the redirect URI configured in Google Cloud Console, or Google Login will fail with a redirect_uri_mismatch error.
- prediction_history.db, prediction_history.csv, and the uploads/ folder are excluded from version control via .gitignore and should be treated as runtime data, not source code.

Exploring the Website's Web Pages:

Home Page — Hero & Live Overview

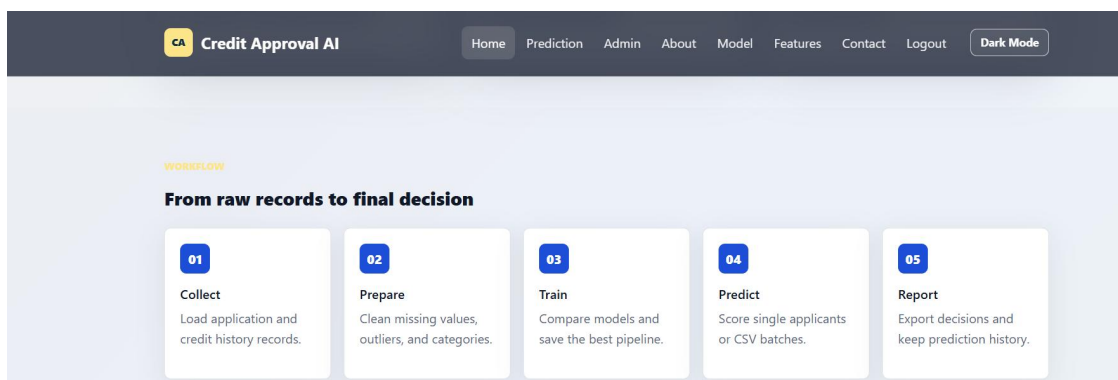
The landing page introduces Credit Card Approval Prediction with a live "Application Desk" summary (saved predictions, approved/rejected counts, batch-ready status) and quick Login / Sign Up / View Model actions.



Home page hero section with live overview panel

Home Page — Workflow

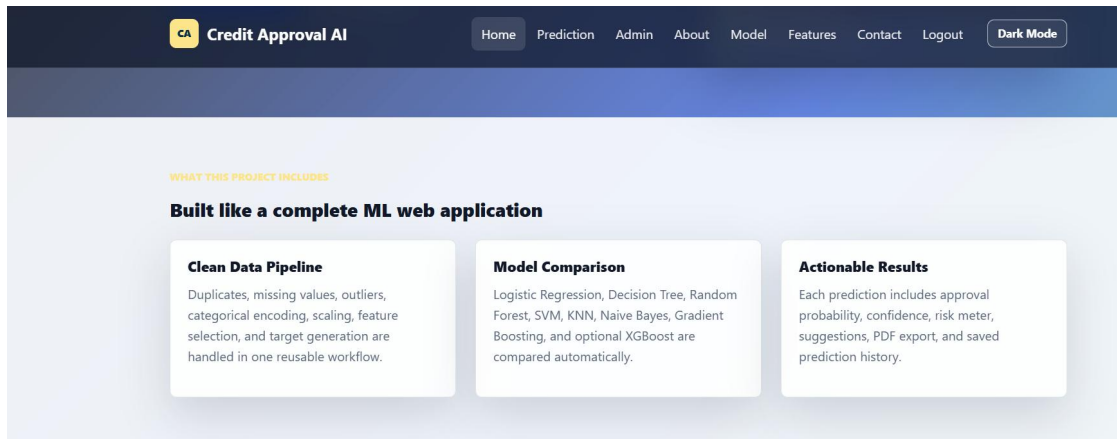
A five-step workflow strip explains the pipeline from raw records to final decision: Collect, Prepare, Train, Predict, and Report.



Workflow section: Collect → Prepare → Train → Predict → Report

Home Page — What This Project Includes

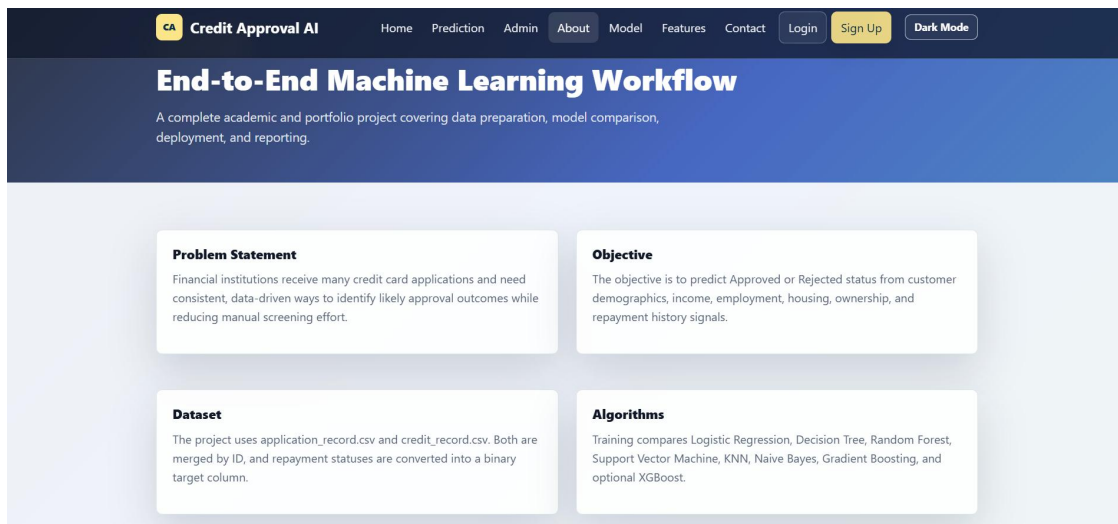
Three highlight cards summarize the clean data pipeline, the model comparison approach, and the actionable results delivered per prediction.



Clean Data Pipeline, Model Comparison, and Actionable Results cards

About Page

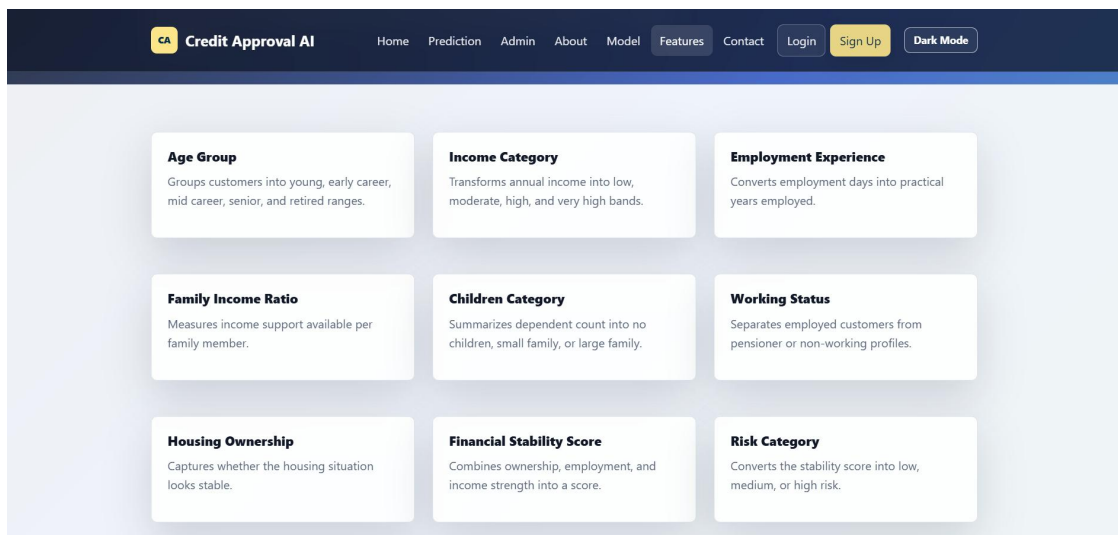
The About page lays out the Problem Statement, Objective, Dataset (application_record.csv and credit_record.csv merged by ID), and the Algorithms compared during training.



About page: Problem Statement, Objective, Dataset, and Algorithms

Features Page

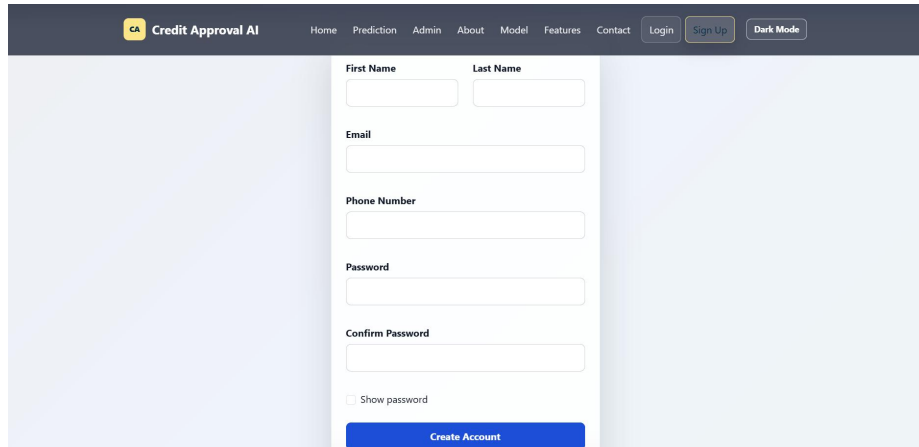
Documents the nine engineered features used by the model, from Age Group and Income Category to the composite Financial Stability Score and Risk Category.



Engineered features surfaced to the end user

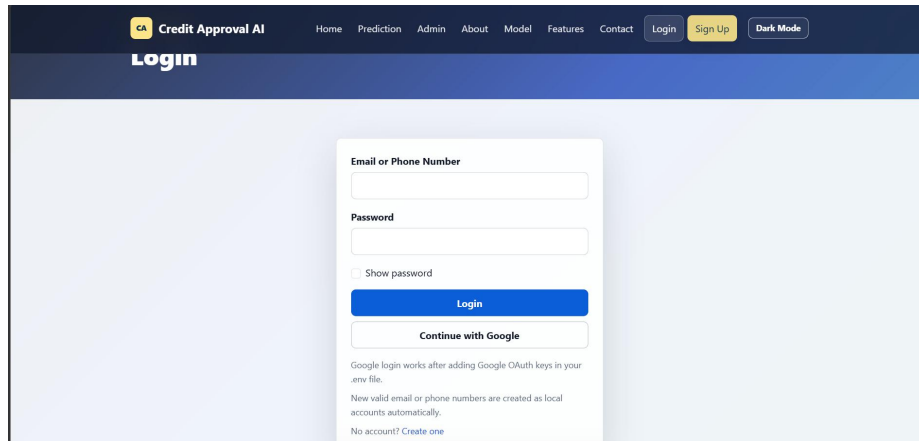
Signup & Login Pages

New users create an account with first/last name, email, phone number, and password. Returning users log in with email/phone and password, or via "Continue with Google".



The screenshot shows the 'Credit Approval AI' web application. The header includes a navigation menu with links: Home, Prediction, Admin, About, Model, Features, Contact, Login, Sign Up, and Dark Mode. The 'Sign Up' button is highlighted. The main content area is a light blue box with a white form. The form has fields for 'First Name', 'Last Name', 'Email', 'Phone Number', 'Password', and 'Confirm Password'. There is a 'Show password' checkbox and a 'Create Account' button at the bottom.

Sign Up page

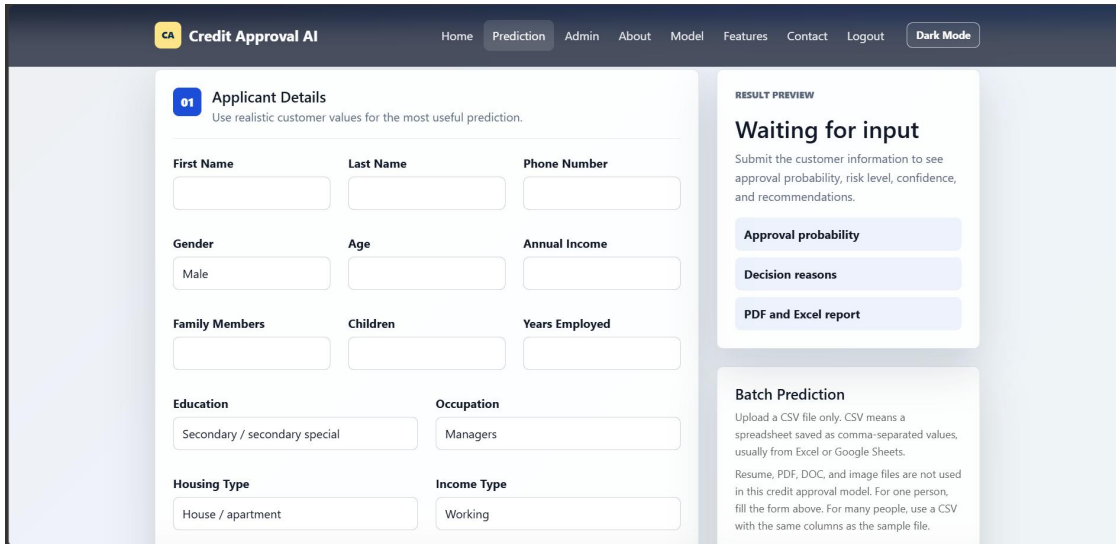


The screenshot shows the 'Credit Approval AI' web application. The header includes a navigation menu with links: Home, Prediction, Admin, About, Model, Features, Contact, Login, Sign Up, and Dark Mode. The 'Login' button is highlighted. The main content area is a light blue box with a white form. The form has fields for 'Email or Phone Number' and 'Password'. There is a 'Show password' checkbox, a 'Login' button, and a 'Continue with Google' button. Below the buttons, there is a note: 'Google login works after adding Google OAuth keys in your .env file. New valid email or phone numbers are created as local accounts automatically. No account? Create one'.

Login page with Google OAuth option

Prediction Page

The core Applicant Details form collects demographic, income, and employment information, with a result-preview panel describing what the applicant will receive (approval probability, decision reasons, PDF/Excel report) and a batch CSV upload option.



The screenshot shows the 'Credit Approval AI' interface. The top navigation bar includes links for Home, Prediction (active), Admin, About, Model, Features, Contact, Logout, and a Dark Mode toggle. The main content area is divided into two sections. The left section, titled '01 Applicant Details', includes a sub-header 'Use realistic customer values for the most useful prediction.' and a form with the following fields: First Name, Last Name, Phone Number, Gender (with 'Male' selected), Age, Annual Income, Family Members, Children, Years Employed, Education (with 'Secondary / secondary special' selected), Occupation (with 'Managers' selected), Housing Type (with 'House / apartment' selected), and Income Type (with 'Working' selected). The right section, titled 'RESULT PREVIEW', contains a 'Waiting for input' message: 'Submit the customer information to see approval probability, risk level, confidence, and recommendations.' Below this are three buttons: 'Approval probability', 'Decision reasons', and 'PDF and Excel report'. At the bottom of the right section is a 'Batch Prediction' section with instructions: 'Upload a CSV file only. CSV means a spreadsheet saved as comma-separated values, usually from Excel or Google Sheets. Resume, PDF, DOC, and image files are not used in this credit approval model. For one person, fill the form above. For many people, use a CSV with the same columns as the sample file.'

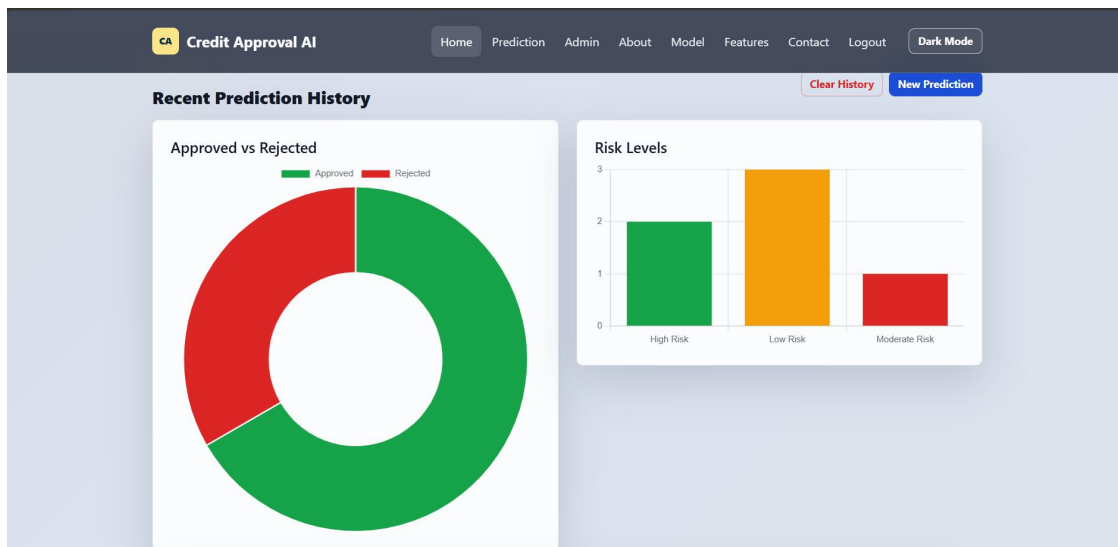
Prediction form with applicant details and batch upload guidance

Prediction History & Analytics

Recent predictions are listed with time, name, income, education, prediction, probability, and risk level, each with a Delete action. Approved-vs-Rejected and Risk Level charts summarize overall trends.

CA Credit Approval AI							
Home Prediction Admin About Model Features Contact Logout Dark Mode							
TIME	NAME	INCOME	EDUCATION	PREDICTION	PROBABILITY	RISK	ACTION
2026-07-03T18:41:28	Harshini N	550000	Higher education	Approved	79.5%	Low Risk	Delete
2026-07-03T18:40:31	Ashwanth ch	250000	Higher education	Approved	57.75%	Moderate Risk	Delete
2026-07-03T18:38:21	Pavan Kumar K	30000	Incomplete higher	Rejected	28.0%	High Risk	Delete
2026-07-03T18:36:11	Sravanthi s	300000	Higher education	Approved	77.5%	Low Risk	Delete
2026-07-03T18:34:25	Maithree P	500	Lower secondary	Rejected	22.0%	High Risk	Delete
2026-07-02T19:08:17	Mahi k	1000000	Secondary / secondary special	Approved	95.0%	Low Risk	Delete

Recent Prediction History table



Approved vs Rejected donut chart and Risk Levels bar chart

Conclusion:

Credit Approval AI is a complete machine-learning web application built with Flask that turns raw applicant and credit-history data into an actionable Approved/Rejected decision. It combines a reusable feature-engineering pipeline, a comparison of multiple classification algorithms, and a user-friendly interface offering single and batch predictions, decision explanations, PDF/Excel export, and an admin analytics dashboard backed by SQLite. Authentication supports both local accounts and Google Sign-In, and the project structure — separate preprocessing, prediction, and Flask layers — keeps the ML logic reusable and the web layer straightforward to extend. Looking ahead, the platform has clear room to grow: richer model explainability using SHAP (already listed as a dependency), scheduled retraining as new credit data arrives, role-based access for admin versus applicant users, and deeper integration between the Model page and live model-comparison reports.